

Heaps - III

Priority queue

↳ is a data structure for maintaining a set S of elements, each with an associated value called the key (or priority)

— A max-priority queue uses a max-heap

Operation: Insert (S, x, k):

insert element x with key k

$$\cong S = S \cup \{x\}$$

— Applications of max-priority queue:

↳ Schedule job in a computer, keeping priorities of jobs

a) once a job is finished, the scheduler selects the highest priority job by

calling Extract-Max

b) New jobs are added with Insert()

Maximum(S): returns the element with largest key

Extract-Max(S): removes & returns element with largest key

Increase-Key (S, x, k): increases value of x 's key to new value k (assumed that $k \geq x.\text{key}$)

Max-Heap-Maximum(A)

if $A.\text{heap-size} < 1$

error "heap underflow"

return $A[1]$

→ $\Theta(1)$

Max-Heap-Extract-Max(A)

$\text{max} = \text{Max-Heap-Maximum}(A)$

$A[1] = A[A.\text{heap-size}]$

$A.\text{heap-size} = A.\text{heap-size} - 1$

$\text{Max-Heapify}(A, 1)$

return max

→ $O(\lg n)$

→ when we implement a priority queue, we treat each array element as the keys to be sorted and assume that the satellite data moves with corresponding keys.

Max-Heap-Increase-Key(A, x, k)

if $k < x.key$

error "New key is smaller than the existing"

$x.key = k$

Find the index i in array A where object x occurs

While $i > 1$ and $A[\text{parent}(i)].key < A[i].key$

exchange $A[i]$ with $A[\text{parent}(i)]$,

(update info that maps priority queue objects to array indices)

$i = \text{parent}(i)$

$O(\lg n)$

+ overhead of mapping priority queue object to indices

Max-Heap-Insert(A, x, n)

if $A.\text{heap-size} == n$

error "heap overflow"

$A.\text{heap-size} = A.\text{heap-size} + 1$

$k = x.key$

$x.key = -\infty$

$A[A.\text{heap-size}] = x$

map x to index heap-size in the array

Max-Heap-Increase-Key(A, x, k)

$O(\lg n)$ + overhead of mapping objects to indices.